

ONLINE SUPPLEMENT

Deep Learning Alpha Signals from Limit Order Books: Practical Insights and Lessons Learned

Main Article Published at Risk.net

Petter N. Kolm and Nicholas Westray

August 5, 2025

Petter N. Kolm is Clinical Professor and Director of the Mathematics in Finance Master’s Program at the Courant Institute of Mathematical Sciences, New York University, NY, USA.

petter.kolm@nyu.edu

Nicholas Westray is a Visiting Researcher in Financial Machine Learning at NYU’s Courant Institute of Mathematical Sciences, New York University, NY, USA.

nicholas.westray@nyu.edu

Overview and Supplementary Material

This online supplement provides supplementary information to the main article, including: (i) a literature review; (ii) descriptions of the deep learning model architectures; (iii) data sources, preprocessing steps, and feature construction; (iv) training and evaluation methodology; and (v) additional implementation notes.

A Brief Literature Review

A large and growing literature develops deep learning models for high-frequency prediction from limit order book (LOB) data. Early work by Tsantekidis et al. (2017) and Tsantekidis et al. (2020) demonstrated the effectiveness of convolutional neural networks (CNNs) for extracting stationary features and improving short-term price prediction. Building on these results, Zhang et al. (2018), Zhang et al. (2019a), Zhang et al. (2019b), Zhang et al. (2021a), and Zhang et al. (2021b) systematically compared deep CNN, LSTM, and Seq2Seq architectures, proposing the influential DeepLOB model that combines convolutional and LSTM layers for robust forecasting performance across LOB datasets.

Other studies have further advanced model architectures. Briola et al. (2020) and Briola et al. (2024) provided comparative analyses and refinements of deep LOB forecasting methods. In the context of multi-horizon and sequence prediction, Zhang et al. (2021b) introduced and benchmarked LSTM-based sequence-to-sequence models and released code for reproducibility. Temporal attention mechanisms were explored by Fan et al. (2019) and Mäkinen et al. (2019), who introduced attention-based networks for multi-horizon prediction and for forecasting jump arrivals, respectively. Bilokon et al. (2023) compared transformer and LSTM architectures for electronic trading, concluding that while attention-based models can add value, simple LSTMs

remain highly competitive for many LOB tasks. Lucchese et al. (2022) confirmed that even modest predictive power, as captured by deep learning in high-frequency settings, can be valuable in practice.

Beyond model selection, several studies emphasize feature engineering and data transformation. Transforming raw LOB states into stationary order flow (OF) and order flow imbalance (OFI) features has proven crucial for robust performance, as shown by Kolm et al. (2023a) and Cont et al. (2014). Notably, Kolm et al. (2023a) found that, within reasonable complexity, the choice of neural network architecture is secondary for short-horizon forecasting: simple LSTM models trained on stationary features can perform nearly as well as more complex networks. This work, along with Cont et al. (2014), Xu et al. (2018), Cont et al. (2023), and Kolm et al. (2023b), provides theoretical justification for using OFI-based features, connecting order flow imbalances at various book levels to future price changes.

Other methodological contributions include the introduction of autoregressive convolutional neural networks for asynchronous time series by Binkowski et al. (2018), the development of recurrent highway networks by Luo et al. (2019), and the release of Keras by Chollet et al. (2015), now standard for deep learning model implementation. Jaisson et al. (2016) modeled stock characteristic time using Hawkes processes, informing the choice of input windows and forecast horizons.

For data handling and microstructure, foundational work by Bouchaud et al. (2018) and others (Abergel et al., 2016; Gould et al., 2013; Bugaenko, 2020) offer comprehensive treatments of LOB mechanics, market impact, and best practices for data preprocessing. Empirical analysis of market impact and event handling is detailed in Bugaenko (2020). On the universality and cross-sectional features of price formation, Sirignano (2019) and Sirignano et al. (2019) find evidence for both universal and model-dependent properties across a broad set of assets.

In summary, this literature provides the empirical, theoretical, and practical context for the present study. References above are cited in full at the end of this supplement.

B Architectures

In this section, we detail the four DNNs used in this article: (i) LSTM, (ii) multihead LSTM, (iii) LSTM Seq2Seq without attention, and (iv) LSTM Seq2Seq with attention.

B.1 Long Short-Term Memory

The key elements of the LSTM are (i) a memory cell, representing its state over time, and (ii) non-linear gates that determine the information flow in and out of the memory cell. It is common to use three gates known as *input*, *output*, and *forget gates* (Gers et al., 2000). The LSTM unit is characterized by the following equations

$$\mathbf{f}_t = \sigma(\mathbf{U}^f \mathbf{x}_t + \mathbf{W}^f \mathbf{h}_{t-1} + \mathbf{b}^f) \quad (1)$$

$$\mathbf{i}_t = \sigma(\mathbf{U}^i \mathbf{x}_t + \mathbf{W}^i \mathbf{h}_{t-1} + \mathbf{b}^i) \quad (2)$$

$$\mathbf{o}_t = \sigma(\mathbf{U}^o \mathbf{x}_t + \mathbf{W}^o \mathbf{h}_{t-1} + \mathbf{b}^o) \quad (3)$$

$$\mathbf{s}_t = \mathbf{f}_t \circ \mathbf{s}_{t-1} + \mathbf{i}_t \circ \tanh(\mathbf{U}^s \mathbf{x}_t + \mathbf{W}^s \mathbf{h}_{t-1} + \mathbf{b}^s) \quad (4)$$

$$\mathbf{h}_t = \mathbf{o}_t \circ \tanh(\mathbf{s}_t), \quad (5)$$

where $\mathbf{x}_t \in \mathbb{R}^N$ is the input vector, $\sigma := (1 + e^{-x})^{-1}$ is the sigmoid activation function, $\mathbf{f}_t \in \mathbb{R}^M$ is the forget gate's activation vector, $\mathbf{i}_t \in \mathbb{R}^M$ is the input gate's activation vector, $\mathbf{o}_t \in \mathbb{R}^M$ is the output gate's activation vector, $\mathbf{h}_t \in \mathbb{R}^M$ is the output vector of the LSTM unit, $\mathbf{s}_t \in \mathbb{R}^M$ is the unit's hidden state vector, and $\mathbf{W} \in \mathbb{R}^{M \times N}$, $\mathbf{U} \in \mathbb{R}^{M \times M}$ and $\mathbf{b} \in \mathbb{R}^M$ (superscripts dropped) are weight matrices and bias vector parameters that are learned during training. Here, $\circ$ represents the element-wise product and $\tanh$ denotes the hyperbolic tangent function. To mitigate the vanishing gradient problem during training, we initialize $\mathbf{b}^f$

to one, following the approach in (Gers et al., 2000). For our regression-based supervised learning problem we feed the output from the LSTM unit through a dense layer with a nonlinear activation function.

The multi-head LSTM we employ in our empirical work uses the same architecture as the LSTM above, with each head having its own LSTM unit and parameter set.

B.2 Sequence to Sequence

Our implementation of the Seq2Seq models is built upon the the code by Zhang et al. (2021b), which draws from the techniques outlined in the works of Cho et al. (2014), Sutskever et al. (2014), and Luong et al. (2015).

A Seq2Seq model consists of (i) an encoder, and (ii) a decoder. The encoder is a *recurrent neural network* (RNN) that sequentially processes an input sequence $\{\mathbf{x}_t\}_{t=1}^T$, where $\mathbf{x}_t \in \mathbb{R}^N$ for all t . At each time step t , the hidden state of the RNN, $\mathbf{h}_t^e \in \mathbb{R}^{M_e}$, is updated according to

$$\mathbf{h}_t^e = f_e(\mathbf{h}_{t-1}^e, \mathbf{x}_t), \quad (6)$$

where f_e is a nonlinear activation function. After processing the final input in the sequence, the encoder outputs the final hidden state, $\mathbf{c} := \mathbf{h}_T^e$, referred to as the *context vector*, providing a summary of the entire input sequence. The decoder, also an RNN, updates its hidden state, $\mathbf{h}_t^d \in \mathbb{R}^{M_d}$, at each time step t , via

$$\mathbf{h}_t^d = f_d(\mathbf{h}_{t-1}^d, \mathbf{y}_{t-1}, \mathbf{c}), \quad (7)$$

and generates the output, $\mathbf{y}_t \in \mathbb{R}^H$, by

$$\mathbf{y}_t = g(\mathbf{h}_t^d, \mathbf{c}), \quad (8)$$

where f_d and g are nonlinear activation functions. For the regression task in this article, we use a dense layer for g .

Seq2Seq models are known to face challenges when dealing with longer sequences. A reason for this is the difficulty in capturing long-term dependencies and summarizing the entire input sequence into a fixed-size context vector. Specifically, the model may struggle to retain important information from earlier parts of the sequence, leading to performance degradation with increasing sequence length.

The Seq2Seq + Attn model by Luong et al. (2015) is designed to effectively handle longer sequences and capture more relevant information from the input sequence during decoding. The attention mechanism allows the decoder to selectively focus on different parts of the input sequence while generating the output sequence (Bahdanau et al., 2014). Rather than using a fixed context vector, the decoder dynamically attends to different segments of the input sequence during each decoding step based on their relevance using the dynamic context vector $\mathbf{c}_t$, given by

$$\mathbf{c}_t := \sum_{s=1}^T \alpha_{t,s} \mathbf{h}_s^e, \quad (9)$$

where the *attention weights*, $\alpha_{t,s} \geq 0$, are given by

$$\alpha_{t,s} := \frac{\exp(\text{score}(\mathbf{h}_{t-1}^d, \mathbf{h}_s^e))}{\sum_{\tau=1}^T \exp(\text{score}(\mathbf{h}_{t-1}^d, \mathbf{h}_\tau^e))}, \quad (10)$$

with the content-based *score* function defined by one of the following three alternatives (Luong et al., 2015)

$$\text{score}(\mathbf{h}_{t-1}^d, \mathbf{h}_s^e) := \begin{cases} (\mathbf{h}_s^e)' \mathbf{h}_{t-1}^d & (\text{dot}), \\ (\mathbf{h}_s^e)' \mathbf{W}_a \mathbf{h}_{t-1}^d & (\text{general}), \\ (\mathbf{v}_a)' \tanh(\mathbf{W}_a [\mathbf{h}_s^e; \mathbf{h}_{t-1}^d]) & (\text{concatenate}), \end{cases} \quad (11)$$

where $\mathbf{v}_a \in \mathbb{R}^{M_a}$ and $\mathbf{W}_a \in \mathbb{R}^{M_a \times (M_e + M_d)}$ are a weight vector and matrix, respectively, learned during training. Then, at each time t , we update the decoder and generate the output, $\mathbf{y}_t$, via

$$\mathbf{h}_t^d = f_d(\mathbf{h}_{t-1}^d, \mathbf{y}_{t-1}, \mathbf{c}_t), \quad (12)$$

$$\mathbf{y}_t = g(\mathbf{h}_t^d, \mathbf{c}_t), \quad (13)$$

where f_d and g are nonlinear activation functions, as above.

C Data

We source our data from LOBSTER (Huang et al., 2011), which offers complete order book state information from Nasdaq, excluding submissions and cancellations of hidden orders. We select all stocks that meet the following criteria from January 1, 2019, through January 31, 2020: (i) membership in the Nasdaq 100 index at some point during the period, (ii) availability of data for at least 260 trading days, and (iii) absence of corporate actions during the study period. We exclude January 9, 2019, due to data issues. This yields a universe of 115 stocks, including well-known names such as Amazon (AMZN), American Airlines (AAL), Facebook (FB), Google (GOOGL), Microsoft (MSFT), and Netflix (NFLX). The larger number of stocks and longer time period compared to most previous studies provide greater confidence in our results.

We preprocess the LOBSTER data to address scenarios such as a single market order executing against multiple limit orders, or a modification to a limit order being processed as a cancellation followed by a new submission (Bouchaud et al., 2018; Bugaenko, 2020). We describe our preprocessing steps in Appendix E.

Our preprocessed dataset, detailed in Table 1 in Appendix E, mirrors the dataset used in Kolm et al. (2023a) and Kolm et al. (2023b), totaling about 10TB uncompressed. Consequently, storage and efficient processing pose significant challenges. Our dataset is one to two orders of magnitude larger than those in similar studies applying DNNs to limit order books. The datasets used by Sirignano (2019) and Sirignano et al. (2019) are exceptions. Their universe, drawn from the S&P 500, is larger than ours. However, the focus of their work differs from ours in that they train classifiers on downsampled time series of order book updates, and Sirignano et al. (2019) investigate whether the forecasting model is universal. It is well-known that not only trades but all changes to the order book are vital in understanding price formation (Cont et al., 2014; Bouchaud et al., 2018). Therefore, as our focus in this article is forecasting alpha term structures rather than questions of universality, and downsampling may result in removing potentially important information, we use all order book updates and train DNNs for each symbol in the dataset.

C.1 Construction of Dependent and Independent Variables

Our stock universe comprises a diverse range of symbols with significantly varying order book activity. For example, EBAY and MSFT exhibit three orders of magnitude difference in terms of the number of order book updates per day. These disparate update frequencies are associated with the concept known as *stock characteristic time*, often formalized by intermittent processes such as Hawkes processes (Jaisson et al., 2016; Bouchaud et al., 2018). To tailor the horizons of the alpha term structure to each

stock, following Kolm et al. (2023a), we use the stock-specific time increment

$$\Delta t := \frac{2.34 \cdot 10^7}{N}, \quad (14)$$

where the numerator represents the number of milliseconds in a trading day, while the denominator, denoted by N , signifies the stock’s average number of non-zero tick-by-tick mid-price returns. This ratio reflects the average waiting time for a change in the mid-price of a specific stock.

Unless explicitly noted otherwise, we select an alpha term structure of dimension $H = 10$, and define our forecast horizons, $\{h_k\}_{k=1}^{10}$, as the multiples of Δt , in particular

$$h_k := \frac{1}{5}k\Delta t, \quad k = 1, \dots, 10. \quad (15)$$

We argue in Kolm et al. (2023a) that this choice of stock-specific forecast horizons adjust the time scale appropriately for each stock. While there is potential to improve the specification of these horizons to consider various intraday effects, this approach allows for sensible comparison of results across stocks. For instance, incorporating intraday profiles is possible within this framework (see, for example, Mertens et al., 2020).

To account for time delays associated with computing model forecasts and round-trip communication between exchanges and trading algorithms, it is standard practice to incorporate latency buffers during alpha development and testing. According to Hasbrouck et al. (2013), using Nasdaq-sourced data for one month in 2007 and 2008, respectively, the fastest traders in their sample have an effective latency of 2–3 milliseconds. Additionally, Zhang et al. (2019a) report that the average time required to produce a return forecast from various state-of-the-art DNNs range from 0.03 to 0.97 milliseconds. To err on the conservative side, we adopt a latency buffer of ten milliseconds in this article.

While Kolm et al. (2023a) consider full order book and OF inputs, in this article we use ten-level OFIs as independent variables, which are reduced representations of order book states, known to be stationary (Cont et al., 2014).

In our training sets, we apply winsorization to all dependent and independent variables at the 0.5% and 99.5% levels, followed by normalization of the resulting data using Z -scores. We retain the normalization parameters obtained from the in-sample data to scale the test and validation sets for out-of-sample evaluation.

D Training and Evaluation

Our training and evaluation involves a quarterly schedule over thirteen months, using a rolling-window with a (1 month, 3 months, 3 months)-structure. The first month is used for validation, the subsequent three months are dedicated to training, and the last three months are reserved for out-of-sample testing. During model training, we minimize the *mean-squared error* (MSE) using *stochastic gradient descent* (SGD) with the Adam optimizer (Kingma et al., 2014). Unless explicitly noted otherwise, we use a relatively small batch size of 256, as evidence suggests that smaller batch sizes lead to local minima that generalize better (Keskar et al., 2017).

When concatenating the time series of order book updates and returns, it is important to not mix data from different trading days within the same batch. To address this issue, we develop a custom Keras generator. To prevent overfitting, we use early stopping for all models, terminating training when the validation loss remains unchanged for five consecutive epochs. We use Python to code our models, leveraging Tensorflow (Abadi et al., 2015) and Keras (Chollet et al., 2015) for implementing the DNNs.

We evaluate the performance of model forecasts at the individual stock level using *out-of-sample* R^2 at horizon h ($R_{OS,h}^2$)

for each test period, defined as

$$R_{OS,h}^2 := 1 - \text{MSE}_{m,h} / \text{MSE}_{\text{bmk},h}, \quad (16)$$

where $\text{MSE}_{m,h}$ and $\text{MSE}_{\text{bmk},h}$ are the mean-squared errors of the model forecasts and benchmark at the h -th horizon, respectively. As a conservative approach, we employ the average out-of-sample return as our benchmark. If $R_{OS,h}^2 > 0$, it indicates that the model outperforms the benchmark. Each forecast relies solely on data available up to the time of its computation. When the time horizon is evident, we omit the subscript h and denote it as R_{OS}^2 .

We train our models on NYU’s Greene¹ and Hudson² high performance computing environments. This infrastructure allows us to massively parallelize the data processing and training of the models. The time to train a single model varies in the range of 10-60 minutes, depending on model and stock.

E Preprocessing of LOBSTER Data

In this section we discuss the format of the LOBSTER data and summarize the preprocessing steps we performed for this article. For additional information we refer to Bouchaud et al. (2018, Appendix A) and Bugaenko (2020, Appendix C).

LOBSTER provides two files for each date and symbol, the message and order book files. The number of rows in each file is identical and the timestamps are given to nanosecond precision. The message file lists every market order arrival, limit order arrival and cancellation. We note that the arrival of hidden orders is not provided. The order book file lists the states of the limit order book with each row corresponding to the state after the corresponding event from the message file. It is the order book file which is the primary object of study in this paper.

Inspection of the message file reveals that there are certain clusters of events that require special care. In particular, consider the following three situations:

- (i) Limit order modification which is implemented as a cancellation followed by an immediate new arrival.
- (ii) Single market order executing against multiple resting limit orders.
- (iii) Auction prints: The auction trade is printed and all the remaining resting limit orders are moved into the limit order book.

Common amongst these situations is that we have one conceptual event which appears as multiple rows in the message file, all with the same timestamp. Consequently, it is critical for modeling purposes that these events are collapsed to avoid artificially inflated prediction accuracy. Therefore, we group the order book data by unique timestamps and take the last entry (they are ordered sequentially within a timestamp by the process which generates this data). This type of processing is standard in the literature using LOBSTER data (see, for example, Bouchaud et al. (2018) and Bugaenko (2020)). In addition, we remove all rows for which the quotes are crossed. As noted in Bouchaud et al. (2018) the occurrence of such events is extremely rare.

Table 1 provides summary statistics after preprocessing the LOBSTER data for the 115 stocks used in this study.

Ticker	Updates (000)	Trades (000)	Price Changes (000)	Price (USD)	Spread (bps)	Volume (USD MM)
AAL	398.13	13.15	10.99	30.65	3.88	50.29
AAPL	1137.36	64.19	127.86	215.80	0.99	1156.30
ADBE	205.29	13.20	41.76	283.83	4.08	194.36
ADI	238.35	10.70	29.23	109.36	3.52	73.59
ADP	128.11	7.82	20.02	160.65	3.62	72.97
ADSK	118.22	8.34	24.27	161.21	5.48	74.96
ALGN	80.86	6.78	20.01	251.14	9.68	81.68
ALXN	93.19	7.47	18.08	117.26	7.45	52.07

Continued on next page

¹Greene has 32K CPU cores, 332 NVIDIA GPUs and 145TB of RAM distributed over 568 nodes.

See, <https://www.nyu.edu/about/news-publications/news/2020/november/GreeneSupercomputer.html>.

²Hudson has 960 CPU cores, 160 AMD GPUs and 10TB of RAM distributed across 20 nodes. See, <https://wp.nyu.edu/connect/2020/06/08/nyu-expands-supercomputing-amd>.

Ticker	Updates (000)	Trades (000)	Price Changes (000)	Price (USD)	Spread (bps)	Volume (USD MM)
AMAT	611.97	18.05	17.15	47.36	2.62	91.72
AMD	1213.04	42.06	16.26	31.33	3.64	343.63
AMGN	137.56	11.06	25.61	198.36	3.85	138.33
AMZN	304.76	31.14	64.23	1795.51	2.59	1531.40
ANSS	63.55	3.37	15.22	206.76	9.39	25.86
ASML	147.63	3.31	21.21	223.92	7.06	46.42
ATVI	423.01	17.97	20.25	49.87	2.81	91.82
AVGO	136.68	12.20	32.10	289.33	4.62	182.39
BIDU	155.90	13.37	26.50	132.58	4.79	129.79
BIIB	88.20	9.80	23.17	265.36	6.01	129.65
BKNG	51.44	4.20	15.00	1881.52	8.68	169.08
BMRN	61.69	5.08	10.81	82.87	10.20	25.70
CDNS	128.56	7.32	15.40	64.99	4.28	32.49
CDW	53.73	4.20	9.73	112.53	7.93	28.02
CERN	117.85	7.57	10.59	66.41	3.53	38.58
CHKP	67.19	3.98	10.82	114.42	6.51	27.00
CHTR	80.97	6.75	19.21	400.68	6.22	111.71
CMCSA	612.78	18.70	7.44	42.40	2.58	128.95
COST	125.41	9.21	26.87	264.93	3.26	120.40
CPRT	80.99	4.77	9.66	73.53	5.03	23.75
CSCO	736.33	23.41	9.81	50.37	2.17	191.94
CSGP	38.27	2.08	9.03	536.05	23.06	34.75
CSX	294.56	12.52	18.20	72.07	2.35	78.23
CTAS	64.05	3.71	14.75	237.20	7.29	34.05
CTSH	271.85	10.22	14.73	64.84	2.39	58.60
CTXS	103.49	7.60	12.32	102.47	3.55	52.46
DISH	138.93	7.27	9.16	34.30	4.95	21.35
DLTR	132.57	8.14	15.42	102.30	4.23	57.93
EA	198.67	13.38	26.25	96.82	4.19	95.90
EBAY	435.54	12.59	5.91	37.05	2.98	66.41
EXC	298.32	7.58	7.21	47.35	2.45	37.55
EXPE	122.44	8.33	16.93	122.62	4.45	62.15
FAST	229.64	8.64	8.99	44.16	3.47	34.60
FB	602.54	44.01	106.38	184.08	1.71	622.29
FISV	178.66	12.38	20.43	97.41	3.14	90.26
FOX	199.41	4.56	9.31	37.81	4.33	19.34
FOXA	239.37	9.02	8.03	38.22	3.39	44.54
GILD	407.04	15.22	20.67	65.53	2.22	96.45
GLIB	47.21	2.06	6.55	61.12	15.97	7.51
GOOG	590.59	13.38	64.96	1206.29	4.71	468.31
GOOGL	517.61	14.61	62.70	1208.90	4.40	445.79
HAS	73.34	5.16	11.42	101.28	6.61	30.45
HOLX	134.48	5.45	7.55	48.02	3.85	20.87
HSIC	99.05	5.18	9.01	66.23	5.41	23.73
IDXX	67.22	3.39	16.85	251.26	9.72	30.08
ILMN	79.51	6.50	20.37	310.12	7.77	84.54
INCY	72.65	5.39	12.91	81.55	8.74	26.16
INTC	966.81	29.58	12.86	52.13	2.10	231.35
INTU	100.24	7.37	23.84	257.26	5.30	88.90
ISRG	78.95	5.84	20.09	535.71	7.83	94.48
JBHT	73.90	4.76	12.57	104.88	7.74	28.07
JD	552.74	18.58	10.95	29.94	3.79	95.11
KHC	351.72	12.51	6.75	32.58	3.48	58.06
KLAC	94.44	7.33	19.12	136.17	6.19	53.08
LBTYA	217.71	5.40	6.73	24.95	5.39	13.11
LBTYK	262.49	6.83	5.15	24.16	4.99	19.60
LILAK	49.89	1.81	3.77	18.09	15.92	2.63
LRCX	154.50	10.43	30.42	215.34	5.48	111.39
LULU	92.51	8.25	19.44	184.49	6.34	87.43
MAR	116.61	6.94	17.30	130.38	4.51	54.67
MCHP	146.21	8.99	19.50	91.11	4.95	51.90
MDLZ	385.19	11.27	6.57	51.78	2.16	68.01
MELI	68.15	3.28	16.99	538.46	19.44	72.08
MNST	207.65	9.02	12.41	59.63	3.14	45.60
MSFT	1314.51	50.80	59.44	132.70	0.96	674.33
MU	1038.48	33.55	23.06	43.55	2.64	224.59
MXIM	165.32	6.96	13.37	57.14	4.29	28.77
MYL	280.72	10.56	6.72	22.25	5.33	30.69
NFLX	299.09	29.56	69.40	329.25	3.92	554.70
NTAP	186.58	9.14	16.83	60.89	4.14	40.97
NTES	59.40	4.19	14.83	268.10	13.15	55.21
NVDA	410.14	32.36	78.34	179.73	3.12	388.83
NXPI	156.79	11.93	21.49	103.13	5.01	88.60
ORLY	71.82	5.06	18.57	393.07	7.90	69.74
PAYX	204.00	7.03	15.93	81.79	2.86	38.72
PCAR	131.38	6.85	13.03	70.63	3.82	34.43
PEP	307.10	13.89	26.13	128.67	1.73	125.57
PYPL	381.85	19.86	39.32	105.65	2.21	164.14
QCOM	547.61	27.86	31.30	73.04	2.23	213.91
QRTEA	121.32	5.23	2.79	13.54	9.75	9.53

Continued on next page

Ticker	Updates (000)	Trades (000)	Price Changes (000)	Price (USD)	Spread (bps)	Volume (USD MM)
REGN	72.53	5.74	18.86	344.93	9.32	70.71
ROST	155.81	8.53	17.27	103.03	3.85	55.34
SBUX	485.99	17.70	19.99	82.04	1.63	139.81
SGEN	44.67	3.95	9.96	82.34	15.15	19.56
SIRI	107.42	4.50	0.29	6.21	16.69	25.20
SNPS	85.39	5.32	15.31	125.01	5.77	35.22
SPLK	84.00	7.54	19.09	129.00	8.36	57.97
STX	226.02	9.21	14.74	50.18	3.67	38.19
SWKS	114.57	8.55	16.98	86.36	5.71	46.25
TMUS	194.31	9.84	15.56	75.70	3.08	60.40
TSLA	283.15	38.43	65.03	292.35	4.66	724.21
TTWO	101.33	7.31	18.13	112.77	6.62	51.14
TXN	324.28	16.38	39.16	116.63	2.43	136.22
UAL	163.59	9.65	17.19	86.65	3.98	59.90
ULTA	77.25	6.45	19.19	297.27	8.64	79.56
VOD	195.93	2.98	0.74	18.56	5.62	14.53
VRSK	71.10	3.96	12.71	142.97	5.62	28.14
VRSN	69.03	4.10	15.10	191.87	6.80	34.27
VRTX	76.14	6.16	17.19	187.81	7.44	58.90
WBA	328.50	13.20	15.18	57.81	2.54	75.19
WDAY	87.49	8.63	20.57	185.12	8.02	86.66
WDC	343.08	17.03	24.69	51.91	3.89	77.10
WLTW	50.77	3.47	11.74	185.22	8.16	27.11
WYNN	90.56	8.39	19.15	123.37	7.61	64.33
XEL	325.18	8.60	10.32	59.35	2.06	44.25
XLNX	200.22	14.14	34.38	107.30	4.31	103.57
XRAY	83.07	5.41	7.11	52.32	4.35	21.67

Table 1: Summary statistics of the 115 stocks from Nasdaq used in this study. The data is sourced from LOBSTER, covering the time period January 1, 2019 through January 31, 2020. Updates, trades, and price changes represent the average number of order book updates, trades and price changes in thousands per day. Price, spread, and volume denote the average price in dollars, bid-ask spread in basis points, and daily volume in dollars, respectively.

References

- Abadi, Martín et al. (2015). *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. URL: <https://www.tensorflow.org/>.
- Abergel, Frédéric, Marouane Anane, Anirban Chakraborti, Aymen Jedidi, and Ioane Muni Toke (2016). *Limit Order Books*. Cambridge University Press.
- Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio (2014). “Neural Machine Translation by Jointly Learning to Align and Translate”. In: *arXiv preprint arXiv:1409.0473*.
- Bilokon, Paul and Yitao Qiu (2023). “Transformers Versus LSTMs for Electronic Trading”. In: *arXiv preprint arXiv:2309.11400*.
- Binkowski, Mikolaj, Gautier Marti, and Philippe Donnat (2018). “Autoregressive Convolutional Neural Networks for Asynchronous Time Series”. In: *International Conference on Machine Learning*. PMLR, pp. 580–589.
- Bouchaud, Jean-Philippe, Julius Bonart, Jonathan Donier, and Martin Gould (2018). *Trades, Quotes and Prices: Financial Markets Under the Microscope*. Cambridge University Press.
- Briola, Antonio, Silvia Bartolucci, and Tomaso Aste (2024). “Deep Limit Order Book Forecasting”. In: *arXiv preprint arXiv:2403.09267*.
- Briola, Antonio, Jeremy Turiel, and Tomaso Aste (2020). *Deep Learning Modeling Of Limit Order Book: A Comparative Perspective*. arXiv: 2007.07319 [q-fin.TR].
- Bugaenko, Anastasia (2020). *Empirical Study of Market Impact Conditional on Order-Flow Imbalance*. arXiv: 2004.08290 [q-fin.CP].
- Cho, Kyunghyun, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio (2014). “Learning Phrase Representations Using RNN Encoder-Decoder For Statistical Machine Translation”. In: *arXiv preprint arXiv:1406.1078*.
- Chollet, François et al. (2015). *Keras*. <https://github.com/fchollet/keras>.

- Cont, Rama, Mihai Cucuringu, and Chao Zhang (2023). “Cross-Impact of Order Flow Imbalance in Equity Markets”. In: *Quantitative Finance*. DOI: DOI : 10.1080/14697688.2023.2236159.
- Cont, Rama, Arseniy Kukanov, and Sasha Stoikov (2014). “The Price Impact of Order Book Events”. In: *Journal of Financial Econometrics* 12.1, pp. 47–88.
- Fan, Chenyou, Yuze Zhang, Yi Pan, Xiaoyue Li, Chi Zhang, Rong Yuan, Di Wu, Wensheng Wang, Jian Pei, and Heng Huang (2019). “Multi-Horizon Time Series Forecasting with Temporal Attention Learning”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2527–2535.
- Gers, Felix A., Jürgen Schmidhuber, and Fred Cummins (2000). “Learning to Forget: Continual Prediction with LSTM”. In: *Neural Computation* 12.10, pp. 2451–2471.
- Gould, Martin D., Mason A. Porter, Stacy Williams, Mark McDonald, Daniel J. Fenn, and Sam D. Howison (2013). “Limit Order Books”. In: *Quantitative Finance* 13.11, pp. 1709–1742.
- Hasbrouck, Joel and Gideon Saar (2013). “Low-Latency Trading”. In: *Journal of Financial Markets* 16.4, pp. 646–679.
- Huang, Ruihong and Tomas Polak (2011). “LOBSTER: Limit Order Book Reconstruction System”. In: *Available at SSRN* 1977207.
- Jaisson, Thibault and Mathieu Rosenbaum (2016). “Rough Fractional Diffusions As Scaling Limits Of Nearly Unstable Heavy Tailed Hawkes Processes”. In: *The Annals of Applied Probability* 26.5, pp. 2860–2882.
- Keskar, Nitish Shirish, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang (2017). *On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima*. arXiv: 1609.04836 [cs.LG].
- Kingma, Diederik P. and Jimmy Ba (2014). *Adam: A Method for Stochastic Optimization*. arXiv: 1412.6980 [cs.LG].
- Kolm, Petter N., Jeremy Turiel, and Nicholas Westray (2023a). “Deep Order Flow Imbalance: Extracting Alpha at Multiple Horizons from the Limit Order Book”. In: *Mathematical Finance* 33.4, pp. 1044–1081.
- Kolm, Petter N and Nicholas Westray (2023b). “Information Content of Cross-Sectional and Multilevel Order Flow Imbalances: A Bayesian Approach”. In: *Available at SSRN* 4568641.
- Lucchese, Lorenzo, Mikko Pakkanen, and Almut Veraart (2022). “The Short-Term Predictability of Returns in Order Book Markets: A Deep Learning Perspective”. In: *arXiv preprint arXiv:2211.13777*.
- Luo, Wei and Feng Yu (2019). “Recurrent Highway Networks With Grouped Auxiliary Memory”. In: *IEEE Access* 7, pp. 182037–182049.
- Luong, Minh-Thang, Hieu Pham, and Christopher D. Manning (2015). “Effective Approaches to Attention-Based Neural Machine Translation”. In: *arXiv preprint arXiv:1508.04025*.
- Mäkinen, Ymir, Juho Kanninen, Moncef Gabbouj, and Alexandros Iosifidis (2019). “Forecasting Jump Arrivals In Stock Prices: New Attention-Based Network Architecture Using Limit Order Book Data”. In: *Quantitative Finance* 19.12, pp. 2033–2050.
- Mertens, Luca, Alberto Ciacchi, Fabrizio Lillo, and Giulia Livieri (2020). “Liquidity Fluctuations And The Latent Dynamics Of Price Impact”. In: *Available at SSRN* 3214744.
- Sirignano, Justin A. (2019). “Deep Learning for Limit Order Books”. In: *Quantitative Finance* 19.4, pp. 549–570.
- Sirignano, Justin A. and Rama Cont (2019). “Universal Features of Price Formation in Financial Markets: Perspectives From Deep Learning”. In: *Quantitative Finance* 19.9, pp. 1449–1459.
- Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le (2014). “Sequence to Sequence Learning with Neural Networks”. In: *Advances in Neural Information Processing Systems* 27.
- Tsantekidis, Avraam, Nikolaos Passalis, Anastasios Tefas, Juho Kanninen, Moncef Gabbouj, and Alexandros Iosifidis (2017). “Forecasting Stock Prices From The Limit Order Book Using Convolutional Neural Networks”. In: *2017 IEEE 19th Conference on Business Informatics (CBI)*. Vol. 1. IEEE, pp. 7–12.

- Tsantekidis, Avraam, Nikolaos Passalis, Anastasios Tefas, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis (2020). “Using Deep Learning For Price Prediction By Exploiting Stationary Limit Order Book Features”. In: *Applied Soft Computing* 93, p. 106401.
- Xu, Ke, Martin D. Gould, and Sam D. Howison (2018). “Multi-Level Order-Flow Imbalance in a Limit Order Book”. In: *Market Microstructure and Liquidity* 04.03n04, p. 1950011.
- Zhang, Zihao, Bryan Lim, and Stefan Zohren (2021a). “Deep Learning For Market By Order Data”. In: *arXiv preprint arXiv:2102.08811*.
- Zhang, Zihao and Stefan Zohren (2021b). “Multi-Horizon Forecasting For Limit Order Books: Novel Deep Learning Approaches And Hardware Acceleration Using Intelligent Processing Units”. In: *arXiv preprint arXiv:2105.10430*.
- Zhang, Zihao, Stefan Zohren, and Stephen Roberts (2018). “BDLOB: Bayesian Deep Convolutional Neural Networks For Limit Order Books”. In: *arXiv preprint arXiv:1811.10041*.
- (2019a). “DeepLOB: Deep Convolutional Neural Networks for Limit Order Books”. In: *IEEE Transactions On Signal Processing* 67.11, pp. 3001–3012.
- (2019b). “Extending Deep Learning Models For Limit Order Books To Quantile Regression”. In: *arXiv preprint arXiv:1906.04404*.